Balatdle Requirements

1. Customer Statement of Requirements

Balatdle is a web-based daily puzzle game. Each day it picks one joker from the card game Balatro, and every player tries to guess which joker it is. The format follows Wordle, and more closely the fan games built on top of it like LoLdle and Valdle.

A player types the name of a joker. The game compares that guess to the hidden answer and gives back color-coded feedback on six attributes: rarity, purchase cost, effect type, activation timing, whether the effect scales, and how the joker is unlocked. A green cell means the attribute matches. A yellow cell means it is close but not exact. Cost also shows an arrow, which tells the player whether the real cost is higher or lower than what they guessed. The player narrows the answer down from those clues. A round should usually take four to six guesses.

Everyone gets the same joker on the same day. That is the point of the format. It gives players something to compare with friends, and it is why the game keeps a win streak and produces a small spoiler-free grid that can be pasted into a chat without giving the answer away.

The game is for people who already play Balatro. There are over a hundred jokers, and the ones that are hard to place from attributes alone are the ones experienced players will enjoy. Newer players can still play, because the feedback teaches the roster as they go.

Three things define the scope of this project.

First, there is no account. A player opens the page and starts guessing. Streak and statistics are saved in the browser and matched to an anonymous record on the server, so a returning player keeps their progress without signing up for anything.

Second, the answer is checked on the server. The joker of the day is never sent to the browser before the puzzle is solved. A player who opens developer tools should not be able to read the answer out of the page.

Third, the following are out of scope for this iteration and are not planned: user accounts and passwords, real-time multiplayer, leaderboards across players, and any guess mode built on Balatro's art or in-game text. The attribute mode needs no game assets, which keeps the project clear of the question of using someone else's artwork.

---

2. Requirements Specification

2.1 Functional Requirements

ID	Requirement
FR-1	The system shall assign exactly one joker as the answer for each calendar date, and shall serve that same joker to every player on that date.
FR-2	The daily puzzle shall roll over at 00:00 UTC, so that the date boundary is the same for all players regardless of their local time zone.
FR-3	The system shall let a player submit a guess by typing a joker name, and shall offer matching joker names as the player types.
FR-4	The system shall reject a submitted name that is not in the joker roster, and shall tell the player the name was not recognized.
FR-5	The system shall evaluate every guess on the server. The answer shall not be included in any response sent to the browser before the puzzle is solved.
FR-6	For each guess, the system shall return feedback on six attributes: rarity, cost, effect type, activation timing, whether the effect scales, and unlock method.
FR-7	Feedback for each attribute shall be one of three results: exact match, partial match, or no match. For cost, the system shall also return whether the answer's cost is higher or lower than the guessed cost.
FR-8	The system shall display every guess the player has made for the current puzzle, with its feedback, so the player can compare guesses.
FR-9	Guesses shall be unlimited. The round ends when the player names the correct joker.
FR-10	When the player guesses correctly, the system shall show a win state that names the answer and reports how many guesses were used.
FR-11	The system shall restore a player's in-progress round if they reload the page or return later the same day.
FR-12	The system shall identify a returning player by a random token stored in the browser, without requiring an account, a password, or an email address.
FR-13	The system shall track each player's current win streak and longest win streak, and shall reset the current streak when the player misses a day.
FR-14	The system shall show a statistics view with games played, games won, win percentage, current streak, and longest streak.
FR-15	The system shall generate a shareable text grid of the player's result that shows the feedback colors and the number of guesses, but not the answer or any joker name.

ID	Requirement
FR-16	The system shall provide an archive of past puzzles that a player can play after the day has ended.
FR-17	The system shall provide a how-to-play panel that explains the six attributes and what each feedback color means.
FR-18	The system shall provide a seed script that loads the joker roster from a data file into the database.

2.2 Non-Functional Requirements

ID	Category	Requirement
NFR-1	Security	The answer joker shall not appear in the page source, in the JavaScript bundle, or in any API response before the player solves the puzzle.
NFR-2	Security	The API shall validate every submitted guess against the roster before it touches the database, and shall use parameterized queries for all database access.
NFR-3	Security	The API shall limit how fast one player token can submit guesses, so the answer cannot be found by submitting the whole roster in a few seconds.
NFR-4	Performance	A guess check shall return in under 300 milliseconds under normal load, measured at the API.
NFR-5	Usability	The game shall be playable in a mobile browser at a width of 360 pixels, with no install and no account.
NFR-6	Accessibility	Feedback shall never be carried by color alone. Every cell shall also carry a text label or an icon, so the game works for a player with color blindness. The game shall be operable with a keyboard.
NFR-7	Reliability	Data rules shall be enforced by the database schema through foreign keys, unique constraints, and value checks, so bad data fails at write time instead of appearing as a wrong clue mid-game.
NFR-8	Maintainability	The joker roster shall live in one data file, so a Balatro balance patch is handled by editing that file and re-running the seed script.
NFR-9	Portability	The application shall run on a local machine with Python, Node, and a PostgreSQL instance, without requiring a paid service.

---

3. Data and Storage Blueprint

3.1 Data Input

There are two sources of data, and they work differently.

The joker roster is read from a file. The roster lives in `jokers.json`, which is committed to the repository. Each entry is one joker, with its name and the six attributes the game compares. A Python seed script reads that file and inserts the rows into the `jokers` table. The application never fetches this data at runtime.

I chose a committed file over scraping the Balatro wiki for three reasons. The roster changes only when the game gets a balance patch, so there is nothing to gain from fetching it on every run. A scraper would make my game depend on a website I do not control, and a layout change on that site would break my game. And a file in the repository is reviewable, which means an attribute that produces a confusing clue can be corrected in one place and seen in the diff.

An example entry looks like this:

```
{
  "name": "Blueprint",
  "rarity": "Rare",
  "cost": 10,
  "effect_type": "Copy",
  "activation": "Passive",
  "scales": false,
  "unlock_method": "Discover 20 jokers"
}
```

Player guesses are entered by hand. A player types a joker name into the guess field in the browser. The frontend suggests matching names as they type, which keeps most guesses free of typos, but the server does not trust that. The API checks the submitted name against the roster before doing anything else. Each accepted guess is written to the `guesses` table as one row.

The daily answers are the third piece of data, and they are written by me rather than by a player. Dates are assigned to jokers ahead of time in the `daily_puzzles` table, so the game does not have to pick a joker at midnight and so the same joker does not come up twice in a short window.

3.2 Database or Storage

As recommended the storage solution is **PostgreSQL**, a relational SQL database. Flask will reach it through the `psycopg` driver.

A relational database is the right shape for this data because The joker roster is fixed in its structure, every joker has exactly the same six attributes, and every query the game runs is a join across keys I already know. A document store would give me flexibility in the shape of a record, which is flexibility this project has no use for.

PostgreSQL specifically, for two reasons. It enforces my data rules for me. Foreign keys, unique constraints, and value checks are written once in the schema, and a bug in my Python cannot write a row that breaks them. It also has real `DATE` and `BOOLEAN` types, which matters here because the puzzle date is the primary key of a table and the scaling flag is a true or false value.

The database holds four tables. The full definitions are in `schema.sql`, and the entity relationship diagram is in `design.md`.

Table	Holds	Key relationships
`jokers`	One row per joker, with the six compared attributes	Referenced by `daily_puzzles` and `guesses`
`daily_puzzles`	One row per date, naming that day's answer	`puzzle_date` is the primary key, so a date can only have one answer. `joker_id` is a foreign key to `jokers`
`users`	One row per anonymous player, keyed by a browser token, with streak and win counts	Referenced by `guesses`
`guesses`	One row per attempt, with the guess number and whether it was correct	Foreign keys to `users`, `daily_puzzles`, and `jokers`

Two design choices in that schema are worth naming. The date is the primary key of `daily_puzzles`, which means the database itself makes it impossible to have two answers for one day. And the streak and win counts are stored on the `users` row rather than recalculated from the `guesses` table, so the statistics view is a single read instead of a scan over every guess the player has ever made.

---

4. Agile Product Backlog

Each story below is also logged as a separate issue in the repository's Issues tab.

Stories marked **MVP** are the first iteration. They are what makes the game playable at all: load the roster, serve a puzzle, take a guess, give feedback, end the round. Stories marked **Backlog** come after that. They make the game worth returning to, but nothing in the core loop depends on them.

The non-functional requirements are not written as stories. Answer secrecy, mobile layout, and color blind support are not features a player asks for, they are conditions every story has to meet. They appear as acceptance criteria on the stories they constrain, and as NFR-1 through NFR-9 above.

US-1: Load the joker roster (MVP)

As the developer, I want the joker roster loaded from a file, so that the game has something to compare guesses against.

- Running the script against an empty database fills the `jokers` table from `jokers.json`
- Running it a second time does not create duplicate rows
- A joker with an invalid rarity fails with a clear error instead of being inserted

Covers FR-18.

US-2: Start today's puzzle (MVP)

As a player, I want today's puzzle when I open the page, so that I can start playing right away.

- The page loads with an empty guess field and no guesses
- Two players opening the page on the same date are working on the same joker
- The puzzle changes at 00:00 UTC, so players in different time zones roll over together
- The answer is not present in the page source, the JavaScript bundle, or any API response
- The board is usable in a mobile browser at a width of 360 pixels

Covers FR-1, FR-2, FR-5.

US-3: Submit a guess by name (MVP)

As a player, I want to guess by typing a joker name, so that I can take a shot at the answer.

- Typing part of a name lists matching jokers
- Selecting a suggestion and submitting records the guess
- A name that is not a real joker is rejected with a message, and does not count as a guess
- The server checks the name against the roster before touching the database

Covers FR-3, FR-4.

US-4: Read the feedback on my guesses (MVP)

As a player, I want color-coded feedback on all six attributes, so that I can narrow down the answer.

- Each guess returns a result for rarity, cost, effect type, activation, scaling, and unlock method
- Each attribute shows as an exact match, a partial match, or no match
- Cost shows an arrow indicating whether the answer is higher or lower
- Every guess so far is listed, in an order that makes clear which came first
- Guesses are unlimited
- Every cell carries a text label or icon as well as a color, and the board works with a keyboard

Covers FR-6, FR-7, FR-8, FR-9.

US-5: Win the round (MVP)

As a player, I want a clear win state when I guess right, so that I know the round is over.

- Guessing the answer ends the round and names the joker
- The number of guesses used is displayed
- The guess field is disabled after a win

Covers FR-10.

US-6: Pick up where I left off (MVP)

As a returning player, I want my round restored when I come back, so that closing the tab does not cost me progress.

- A first visit creates a token in the browser and a matching record on the server
- Reloading mid-round restores every guess and its feedback
- Returning after a win shows the finished state, not a fresh puzzle
- No email address, password, or personal information is collected

Covers FR-11, FR-12.

US-7: See my streak and statistics

As a player, I want my streak and win record, so that I have a reason to come back tomorrow.

- Solving on consecutive days increases the current streak, and missing a day resets it
- The longest streak is kept even after the current one resets
- Games played, games won, and win percentage are shown alongside both streaks
- The view loads as a single request, without recalculating from every past guess

Covers FR-13, FR-14.

US-8: Share my result without spoiling it

As a player, I want to copy my result as a grid, so that I can compare with friends without spoiling it.

- The grid shows the feedback colors and the number of guesses
- No joker name appears anywhere in the copied text
- One button copies it to the clipboard

Covers FR-15.

US-9: Play puzzles I missed

As a player, I want past puzzles kept around, so that a day I missed is not lost.

- Past dates are listed and can be opened
- An archive round plays the same way as the daily one
- Archive results do not change the daily streak

Covers FR-16.

US-10: Learn the rules

As a new player, I want the rules explained, so that my first guess is not blind.

- A how-to-play panel explains all six attributes
- It explains the difference between an exact match and a partial match
- It is reachable at any time, not just on the first visit

Covers FR-17.